

# Copilot Features and Context

## How It Thinks

---

*What Copilot can see — and what it cannot. Plus the instructions you can give it.*

Inline context

Chat context

#-mentions

Semantic search

Custom instructions

Sandeep Soni · Deccansoft Software Services Pvt Ltd

Deck 3 · GH-300 Certified GitHub Copilot Expert Training

# The context principle — what the model can see, period

*"The model has no memory of previous sessions and no access to files it has not been given. Everything it knows comes from the context assembled for the current request."*

## Inline completions

Copilot builds a focused context from your cursor area, open tabs, file paths, comments, and repo structure. It picks what is most relevant — it does NOT load the full codebase.

### Optimised for:

*Speed and flow. Tab-triggered. 200–500 ms.*

## Chat & Agent mode

A much richer context: conversation history, semantic search across your repo, explicit references, tool outputs, and web content. Reaches far beyond open files.

### Optimised for:

*Reasoning and multi-file work. 1–5 s; turns; iterative.*

# Inline completions — the six context sources

## 1 Cursor position

Code above and below your cursor — the primary signal.

## 2 Comments & docs

JSDoc, docstrings, inline comments — high-signal prompts.

## 3 Open tabs

Types, functions, patterns from files currently open.

## 4 File path / language

Extension and folder infer context (user.test.ts → testing).

## 5 Fill-in-the-middle

Considers preceding AND following code — not only above.

## 6 Repo structure

Directory tree and file names — but NOT file contents unless open.

# Inline completions — triggers and shortcuts

## Automatic triggers

- New line after a comment → suggestion.
- Function signature → body suggested.
- Opening brace { → content suggested.
- Typing pause (200–300 ms) → triggers suggestion.
- After keyword (if, for, return) → completion follows.
- Incomplete statement (const users =) → value.
- **Manual trigger: Alt + \ (mac: Option + \).**

## Keyboard shortcuts

Accept entire suggestion

**Tab**

Accept word-by-word

**Ctrl + →**

Reject suggestion

**Esc**

Next alternative

**Alt + ]**

Previous alternative

**Alt + [**

Trigger manually

**Alt + \**

# Next Edit Suggestions — Copilot predicts your next change

## How NES works

- Analyses your recent editing history.
- Predicts what you will want to change next.
- Shows an arrow in the gutter at the predicted spot.
- Tab jumps to the suggestion location.
- Tab again accepts the edit.
- Works across the file — not only at the cursor.

## Best for

- Refactoring — renaming a symbol across many spots.
- Data structures — add a field and update every reference.
- Code + comments + tests staying in sync after one edit.
- Bug fixes — identifies related fixes after your initial change.
- Missing imports — detects and adds imports after new code.

# When Chat earns its keep — five categories of task

## Understanding

- Explain unfamiliar code
- Learn new libraries
- Understand design patterns
- Clarify language concepts

## Debugging

- Find bugs in code
- Fix errors with /fix
- Optimise slow queries
- Trace unexpected behaviour

## Architecture

- Explore framework options
- Design API structure
- Evaluate trade-offs
- Get recommendations

## Refactoring

- Improve readability
- Extract reusable functions
- Modernise to async/await
- Add error handling

## Docs & Tests

- Generate tests (/tests)
- Write JSDoc (/doc)
- Create usage examples
- Generate README

# The mode catalog — four ways Copilot Chat behaves

## Ask

Conversational Q&A. Reads but never modifies files. Best for explain, learn, decide.

## Agent

Autonomous multi-step execution. Plans, edits files across the project, runs commands, iterates on errors.

## Plan

Designs a detailed plan first; you review and approve before any code is written. Best for large features.

## Custom

Specialised personas via `.agent.md` — Planner, Reviewer, Doc-Writer, Security Auditor, etc. (covered in Deck 10).

**Pick by intent:** Ask when you want to learn, Plan when you want a reviewable design, Agent when you want it done, Custom when the team has agreed how a role should behave.

# Chat & Agent mode — six layers of context, each turn

|   |                             |                                                                               |
|---|-----------------------------|-------------------------------------------------------------------------------|
| 1 | <b>Your message</b>         | The current prompt you type.                                                  |
| 2 | <b>Conversation history</b> | All prior turns — auto-summarised when the window fills.                      |
| 3 | <b>Implicit context</b>     | Active file, selection, visible errors, git state — included automatically.   |
| 4 | <b>Explicit references</b>  | #file, #codebase, #terminalSelection, @workspace, drag-and-drop, #fetch URLs. |
| 5 | <b>Semantic search</b>      | Repo index finds relevant code by meaning — even files you have not opened.   |
| 6 | <b>Tool outputs</b>         | In agent mode: file reads, terminal commands, lint errors, test runs.         |

*In agent mode the loop runs autonomously — it reads files, runs commands, observes errors, and iterates until the goal is met.*



# Context variables — type # to reference precise context

| Variable              | What it attaches                               |
|-----------------------|------------------------------------------------|
| # file                | A specific file from your workspace.           |
| # selection           | The currently highlighted text in the editor.  |
| # editor              | The entire file currently open in the editor.  |
| # codebase            | Semantic search across the whole workspace.    |
| # problems            | IDE diagnostics — lint, compiler, type errors. |
| # terminalLastCommand | The last command run in the VS Code terminal.  |
| # terminalSelection   | Text currently selected in the terminal.       |
| # fetch               | Pull live content from a URL into context.     |

# Slash commands — built-in actions on a / prefix

| Command               | What it does                                    | Example                                                                   |
|-----------------------|-------------------------------------------------|---------------------------------------------------------------------------|
| <code>/clear</code>   | Start a new chat session                        | <code>/clear</code> — resets context                                      |
| <code>/explain</code> | Explain the code in the active editor           | <code>/explain</code> how does the middleware pipeline work in this file? |
| <code>/fix</code>     | Propose a fix for problems in the selected code | <code>/fix</code> the null reference in the selected method               |
| <code>/new</code>     | Scaffold a new project or file                  | <code>/new</code> create a Python FastAPI project with a todo endpoint    |
| <code>/tests</code>   | Generate tests for the selected code            | <code>/tests</code> generate xUnit tests for the selected actions         |
| <code>/doc</code>     | Generate docs for the selected code             | <code>/doc</code> #selection — include @example tags                      |
| <code>/help</code>    | Quick reference and basics                      | <code>/help</code> — shows what is available here                         |

# Power-user combinations — slash + # together

## Scoped testing

prompt

```
/tests # selection — use Jest;  
include null+ empty+ max-int  
and parameterise boundaries.
```

## Multi-file comparison

prompt

```
Compare # file:oldAuth.js with  
# file:newAuth.js and explain  
every behaviour that changed.
```

## Debug with terminal

prompt

```
The test in # selection is failing  
with # terminalSelection error.  
What's the root cause + fix?
```

## Typed documentation

prompt

```
/doc # selection with usage  
examples that reference  
# file:types.ts interfaces.
```

# Chat participants — @-mentions for specialised context

## @ workspace

- "Where is authentication handled?"
- "Create a README for this project"
- "What external APIs do we use?"

## @ terminal

- "Why did my build fail?"
- "How to run tests in watch mode?"
- "What does this error mean?"

## @ vscode

- "How to change font size?"
- "Find a linter for Python"
- "Shortcut for multi-cursor"

## @ github

- Search repos, issues, PRs
- Code search across the org
- Cross-link with MCP-enabled tools

**Custom participants** — extensions can register their own @handles (e.g. @prisma, @azure). Discover them the same way you discover slash commands: type @ in chat.

# The context window — finite, even when it looks big

*Even models with 128K–200K+ token windows do not load your entire repo. Copilot strategically selects what is most relevant. When the window fills, older history is auto-summarised; you can also type `/compact` to trigger it.*

## File length

Long files consume the window quickly. Keep modules at 300–500 lines.

## Open files count

Too many open tabs dilute context.  
Close unrelated files.

## Comment quality

Strategic comments guide suggestions. Focus on intent, not description.

## Code complexity

Nested / verbose code consumes more tokens than clean, structured code.

## Chat history len

Long conversations lose early details when summarised. Start fresh.

## Model choice

Different models have different windows and reasoning. Match to task.

## Repository indexing & semantic search — finding code by meaning

### Auto-index

When you open Copilot Chat, your repo is indexed in the background. Initial up to 60 s; re-indexing is near-instant.

### Vector embed

Code is converted to vector embeddings that capture meaning. Two functions with different names but the same concept match.

### Smart retrieval

When you prompt, Copilot finds the most semantically relevant code and adds it to the request — no manual #file needed.

### Key facts

- Built from the default branch — main / master.
- Shared across all team members; works in github.com, VS Code, and JetBrains.
- No limit on repos; agent mode uses it automatically.

# What Copilot CANNOT see — your blind spots

## Secrets & credentials

API keys, passwords — blocked by safety filters.

## Network / runtime state

Live API responses, runtime variables, server state.

## Build artifacts

Compiled output, images, PDFs — not indexed.

## Private packages

Internal registries, proprietary library internals.

## Config outside project

System-level configs, global settings files.

## Full git history

Recent changes — yes; the whole commit log — no.

## Environment variables

Unless referenced in code or comments.

## Database schemas

Unless represented in code (e.g. ORM models).

**Workaround:** provide these explicitly — via comments, attached files, #fetch URLs, or — covered next — custom instructions and AGENTS.md.

# Optimising context — practical levers in both surfaces

## Inline completions

- Keep files modular (300–500 lines) so Copilot can see the whole file.
- Close irrelevant tabs — be intentional about your workspace.
- Use clear function names — `calculateCompoundInterest` beats `calc`.
- Write targeted comments — describe intent, not behaviour.
- Position the cursor near the relevant code.
- Switch to Chat when inline context is not enough.

## Chat & Agent mode

- Use `#`-mentions explicitly — reference `#file:auth.ts` instead of describing it.
- Start fresh threads for new tasks — stale history confuses Copilot.
- `/compact` for long sessions — optionally add focus guidance.
- Be specific, not vague — quote the error, name the file.
- `#fetch` latest docs to avoid stale training-data patterns.
- In agent mode, give the goal — let it explore. Do not micro-prompt.



# Custom Instructions

# Custom Instructions — teach Copilot your project's conventions

*A persistent, version-controlled answer to "what does this team consider good code?" — added to every Copilot request automatically. The single highest-leverage knob for suggestion quality.*

## Markdown files

Plain .md files describing conventions, tech stack, architecture, and "do / do not" rules.

## Better suggestions

Copilot generates more relevant code by understanding your stack and patterns from the start.

## Repo or path-specific

Apply org-wide, repo-wide, or per-path via glob patterns. Layer them.

## Version-controlled • shared

Live in the repo. Reviewed in PRs. Automatically picked up by every team member.

# Three layers — repo-wide, path-specific, and AGENTS.md

## Repository-wide `.github/copilot-instructions.md`

- Auto-loaded for any work in the repo.
- Apply to all files unless overridden.
- Single file for project-wide standards.
- Soft cap ~1,000 lines for best results.

## Path-specific `.github/instructions/ *.instructions.md`

- Scoped by glob via `applyTo` frontmatter.
- `frontend.instructions.md` — React rules.
- `backend.instructions.md` — API rules.
- Layer with the repo-wide file — they add, not replace.

## AGENTS.md `AGENTS.md` (repo root)

- Open cross-tool standard.
- Honored by Copilot, Claude Code, Cursor, and other AI tools.
- Read alongside `copilot-instructions.md` — they layer.
- Human-readable first — onboarding and AI in one file.

## Setting up custom instructions — three places, one source of truth

### Organization

- github.com → Org Settings → Copilot → Custom Instructions
- Free-form text box, organization-wide
- Applies in Chat, Code Review, Cloud Agent
- Useful for compliance / security defaults

### Repository

- Create .github/copilot-instructions.md
- Markdown — standards, patterns, examples
- Version-controlled, shared with team
- Works in VS Code, Visual Studio, JetBrains

### VS Code / IDE

- Ctrl/Cmd , → "instruction file"
- Enable / disable per-workspace
- Custom user-level files for personal style
- Verify load via the References list in Chat

# Custom instructions in practice — what to put in the file

`.github/copilot-instructions.md`

```
# Copilot instructions

## Stack
Python 3.12 + FastAPI + SQLAlchemy + pytest
Frontend: React 18 + TypeScript 5 + Vite

## Conventions
- snake_case for Python, camelCase for TS
- Result<T, E> pattern; no raw exceptions
- All endpoints async; CancellationToken-aware

## Architecture
- Repository pattern; thin controllers
- DTOs at the boundary, entities internal

## Do
- Use match/case for state transitions
- Add @pytest.mark.parametrize for boundary tables

## Do NOT
- Add new deps without justification
- Use print(); use the structured logger
- Catch broad Exception in production paths
```

## Good content

- Tech stack with versions.
- Project-specific patterns (Result type, Repository).
- Naming + style decisions that differ from defaults.
- Explicit do / do-not list — bans are as useful as encouragements.

## Avoid

- Re-stating what is already in the code (file names, line counts).
- Style points handled by Prettier / Black / dotnet-format.
- Aspirational instructions you do not actually follow — drives drift.
- Secrets, tokens, or PII — these live in env / vault, not instructions.

# Limitations and the meta-prompt — write better instructions

## Limitations to know

- Instructions are suggestions, not guarantees.
- Complex rules may be ignored for simple tasks.
- Too many instructions reduce suggestion quality.
- Code Review instructions capped at 4,000 characters.
- Keep repo instructions under ~1,000 lines.

## Validate that they loaded

- Open the References list in any Chat response.
- Look for copilot-instructions.md / AGENTS.md / \*.instructions.md entries.
- If missing, check VS Code "instruction file" setting + repo location.
- Path-specific files only fire when applyTo matches.

meta-prompt

# Meta-prompt — get Copilot to draft your instructions file

Role: Senior developer with enterprise experience.

Task: Generate github/copilot-instructions.md for a Python + React project.

Backend (FastAPI) and Frontend (React+ TS) live in the same workspace.

Split: a base file at the root + project-specific files under github/instructions/.

Style: divided into sections (Stack · Conventions · Architecture · Do · Do NOT).

Limit: 2,000 words. Ask clarifying questions before producing the final file.